

Skill 10

Task 1

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>
// Signal handler using sigaction
void signalHandler(int signo, siginfo_t *info, void *context)
{
    printf("\n-----\n");
    printf("Signal Received: %d\n", signo);
    switch (signo)
    {
        case SIGINT:
            printf("Handled SIGINT (Ctrl+C)\n");
            break;
        case SIGTERM:
            printf("Handled SIGTERM\n");
            break;
        case SIGUSR1:
            printf("Handled SIGUSR1\n");
            break;
        default:
            printf("Other Signal\n");
    }
    printf("Sender PID: %d\n", info->si_pid);
    printf("Signal Code: %d\n", info->si_code);
    // Context is available here
    printf("Signal context captured successfully.\n");
    printf("-----\n");
}
```

```

}
int main()
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    // Register handler
    sa.sa_sigaction = signalHandler;
    // Configure signal actions
    sa.sa_flags = SA_SIGINFO;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);
    printf("Signal handlers registered.\n");
    printf("PID = %d\n", getpid());
    printf("\nTesting signal events...\n");
    printf("1. Press Ctrl+C (SIGINT)\n");
    printf("2. Run: kill -SIGUSR1 %d\n", getpid());
    printf("3. Run: kill -SIGTERM %d\n", getpid());
    printf("Press Ctrl+Z to stop the program.\n\n");
    // Verify stability by continuously running
    while (1)
    {
        printf("Program is running...\n");
        sleep(5);
    }
    return 0;
}

```

Output:

```

root@Tejasri:~# ./signal_demo
Signal handlers registered.
PID = 612

Testing signal events...
1. Press Ctrl+C (SIGINT)
2. Run: kill -SIGUSR1 612
3. Run: kill -SIGTERM 612
Press Ctrl+Z to stop the program.

Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^Z
[1]+  Stopped                  ./signal_demo
root@Tejasri:~# |

```

Task 2

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```

GNU nano 7.2 shell_signal.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
pid_t fg_pid = -1;
// Parent handles Ctrl+C
void sigint_handler(int sig)
{
    printf("\nParent received SIGINT (Ctrl+C)\n");
    if (fg_pid > 0)
    {
        printf("Forwarding SIGINT to foreground process (PID %d)\n", fg_pid);
        kill(fg_pid, SIGINT);
    }
    else
    {
        printf("No foreground process running.\n");
    }
}
int main()
{
    signal(SIGINT, sigint_handler);
    fg_pid = fork();
    if (fg_pid < 0)
    {
        perror("fork");
        return 1;
    }
}

```

```

// Child process
signal(SIGINT, SIG_DFL);
printf("Foreground job started.\n");
printf("Child PID: %d\n", getpid());
printf("Press Ctrl+C to interrupt the child.\n");
while (1)
{
    printf("Foreground job is running...\n");
    sleep(2);
}
else
{
    int status;
    printf("Shell PID: %d\n", getpid());
    printf("Foreground PID: %d\n", fg_pid);
    waitpid(fg_pid, &status, 0);
    if (WIFSIGNALED(status))
    {
        printf("\nForeground job terminated by signal %d\n",
            WTERMSIG(status));
    }
    else if (WIFEXITED(status))
    {
        printf("\nForeground job exited normally.\n");
    }
    fg_pid = -1;
    printf("Job state updated: Finished\n");
    printf("Shell is still running.\n");
}

```

Output:

```
root@Tejasri:~# nano shell_signal.c
root@Tejasri:~# gcc shell_signal.c -o shell_signal
root@Tejasri:~# ./shell_signal
Shell PID: 656
Foreground PID: 657
Foreground job started.
Child PID: 657
Press Ctrl+C to interrupt the child.
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
^C
Parent received SIGINT (Ctrl+C)
Forwarding SIGINT to foreground process (PID 657)

Foreground job terminated by signal 2
Job state updated: Finished
Shell is still running.
root@Tejasri:~# |
```

Task 3

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```
GNU nano 7.2 sigstp_demo.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
pid_t child_pid;
int job_suspended = 0;
// Handle Ctrl+Z (SIGTSTP)
void sigtstp_handler(int sig)
{
    if (child_pid > 0)
    {
        printf("\nShell: Captured SIGTSTP (Ctrl+Z)\n");
        // Suspend the child process
        kill(child_pid, SIGSTOP);
        job_suspended = 1;
        printf("Shell: Foreground job suspended.\n");
        printf("Shell: Job table updated (Stopped).\n");
        printf("Shell: State preserved.\n");
        printf("Use 'fg' simulation to resume.\n");
    }
}
int main()
{
    signal(SIGTSTP, sigtstp_handler);
    child_pid = fork();
    if (child_pid < 0)
    {
        perror("fork");
    }
}
```

```

GNU nano 7.2 sigstp_demo.c *
{
    printf("Foreground Job Started\n");
    printf("Child PID: %d\n", getpid());
    while (1)
    {
        printf("Child is running...\n");
        sleep(2);
    }
}
else
{
    sleep(5);
    printf("\nPress Ctrl+Z to suspend the child process.\n");
    while (1)
    {
        if (job_suspended)
        {
            char choice;
            printf("\nResume suspended job? (y/n): ");
            scanf("%c", &choice);
            if (choice == 'y' || choice == 'Y')
            {
                printf("Shell: Resuming job...\n");
                kill(child_pid, SIGCONT);
                job_suspended = 0;
                printf("Shell: Job table updated (Running).\n");
            }
        }
        else
        {

```

Output:

```

root@Tejasri:~# ./sigstp_demo
Foreground Job Started
Child PID: 706
Child is running...
Child is running...
^ZChild is running...

Shell: Captured SIGTSTP (Ctrl+Z)
Shell: Foreground job suspended.
Shell: Job table updated (Stopped).
Shell: State preserved.
Use 'fg' simulation to resume.

Press Ctrl+Z to suspend the child process.

Resume suspended job? (y/n): y
Shell: Resuming job...
Shell: Job table updated (Running).
Child is running...
Child is running...
^Z
Child is running...
Shell: Captured SIGTSTP (Ctrl+Z)
Shell: Foreground job suspended.
Shell: Job table updated (Stopped).
Shell: State preserved.
Use 'fg' simulation to resume.

Resume suspended job? (y/n): |

```

Task 4

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <termios.h>
int main()
{
    pid_t pid;
    int status;
    pid_t shell_pgid;
    /* Ignore SIGTTOU so parent can safely change terminal control */
    signal(SIGTTOU, SIG_IGN);
    shell_pgid = getpgrp();
    printf("Shell PID : %d\n", getpid());
    printf("Shell PGID: %d\n", shell_pgid);
    pid = fork();
    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    if (pid == 0)
    {
        /* Child creates its own process group */
        setpgid(0, 0);
        printf("\nChild Process Created\n");
        printf("Child PID : %d\n", getpid());

```

```

        printf("Child exiting normally.\n");
        exit(0);
    }
    else
    {
        /* Put child in its own process group */
        setpgid(pid, pid);
        sleep(1);
        printf("\nParent transferring terminal control to child...\n");
        if (tcsetpgrp(STDIN_FILENO, pid) == -1)
            perror("tcsetpgrp");
        /* Wait for child */
        waitpid(pid, &status, 0);
        printf("\nRestoring terminal control to parent...\n");
        if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
            perror("tcsetpgrp");
        printf("\nValidation:\n");
        printf("✓Process group created.\n");
        printf("✓Group ID assigned.\n");
        printf("✓Terminal ownership transferred.\n");
        printf("✓Terminal ownership restored.\n");
        printf("✓Parent and child coordinated correctly.\n");
        if (WIFEXITED(status))
            printf("Child exited with status %d\n", WEXITSTATUS(status));
        printf("Program completed successfully.\n");
    }
    return 0;
}

```

Output:

```
root@tejasri:~# ./groups
Shell PID : 763
Shell PGID: 763

Child Process Created
Child PID : 764
Child PGID: 764
Running for 10 seconds...
Child running... 1

Parent transferring terminal control to child...
Child running... 2
Child running... 3
Child running... 4
Child running... 5
Child running... 6
Child running... 7
Child running... 8
Child running... 9
Child running... 10
Child exiting normally.

Restoring terminal control to parent...

Validation:
✓Process group created.
✓Group ID assigned.
✓Terminal ownership transferred.
✓Terminal ownership restored.
✓Parent and child coordinated correctly.
Child exited with status 0
Program completed successfully.
root@Tejasri:~# |
```